

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

AI Prompt Library for Automation Engineers

Ready-to-use prompts for writing, debugging and improving Playwright tests.

Introduction

AI is a powerful pair-programmer for automation — but only when you prompt it well and verify the output. This library gives you ready-to-use prompts grouped by task, plus the safety rules that keep your use professional.

How to Use This Library

- Copy a prompt and replace the [bracketed] parts with your context.
- Always review and run the generated code — never trust it blindly.
- Never paste secrets, credentials or production data.
- Treat AI output as a draft you own and verify.

Writing Tests

1. Act as a senior automation engineer. Write a Playwright test in TypeScript for [behaviour]. Use getByRole/getByLabel locators and web-first assertions. Do not use waitForTimeout.
2. Generate a Playwright test for the login flow: valid login, invalid password, and empty fields. Use the AAA pattern and clear test names.
3. Convert these manual test steps into a Playwright test, keeping one behaviour per test: [paste steps].
4. Write a data-driven Playwright test that runs the same flow for this set of inputs: [paste data].

Locators

1. Given this HTML, suggest the most robust Playwright locator using role/label/text where possible, avoiding brittle CSS: [paste HTML].
2. Review these locators and flag any that are brittle (CSS chains, nth-child, indexes). Suggest user-facing alternatives: [paste locators].
3. Explain why getByRole is preferred over CSS selectors, with two concrete examples.

Page Objects & Structure

1. Refactor this Playwright test into a Page Object Model. Keep locators and actions in the class and assertions in the test: [paste test].
2. Suggest a folder structure and naming convention for a growing Playwright suite covering auth, checkout and account features.
3. Create a reusable custom fixture that provides a logged-in page for my tests.

Debugging & Flakiness

1. This Playwright test is flaky. Review it for timing issues, missing awaits, shared state and brittle locators, and suggest fixes: [paste test].
2. Explain what this Playwright error means and how to fix it: [paste error].
3. Suggest how to make this test reliable without using retries or waitForTimeout: [paste test].

API & Data

1. Write a Playwright API test using the request fixture that creates a resource and asserts the status and key response fields for: [endpoint + payload].
2. Show how to use an API call to set up state before a UI test, so the UI test only covers the behaviour under test.
3. Generate a factory function that produces unique, valid test data for [entity], ensuring no collisions between test runs.

CI/CD & Config

1. Write a GitHub Actions workflow that installs dependencies and browsers, runs Playwright tests, and uploads the report on failure.
2. Review my playwright.config.ts and suggest improvements for reliability and speed: [paste config].
3. Explain how to configure traces and screenshots so CI failures are easy to diagnose.

Learning & Explaining

1. Explain Playwright auto-waiting simply, then give a before/after example replacing a manual wait with an assertion.
2. Explain the test pyramid and where Playwright UI and API tests fit, with a QA example.
3. Quiz me with five questions on Playwright locators and assertions, then mark my answers.

Safety Rules

- No secrets, credentials, customer or production data in prompts.
- Run and review every generated test before committing.
- Verify locators and assertions against the real app.
- You own the code — AI drafts, you decide.

Common Mistakes

- Committing AI code without running it.

- Accepting brittle locators it suggests.
- Pasting sensitive data or proprietary code into public tools.

INSIDE STLC PRO TIP

AI accelerates automation, but reviewing what it writes is where your engineering judgement shows. The best engineers prompt fast and verify hard.